

Arrays and Their Limitations

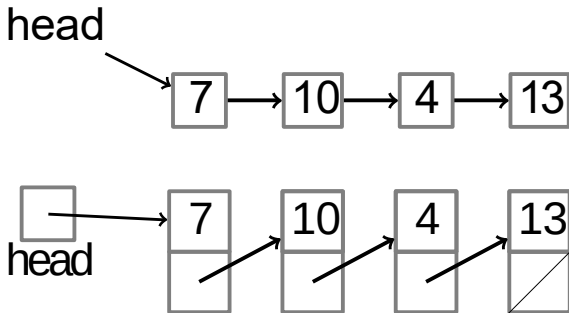
A collection of elements stored in contiguous memory locations, accessed via index.

	Add	Remove
Beginning	$O(n)$	$O(n)$
End	$O(1)$	$O(1)$
Middle	$O(n)$	$O(n)$

Fixed size, inefficient insertion and deletion operations (since elements need to be shifted).

linked list overcomes some limitations of arrays. It is a dynamic data structure that can grow and shrink as needed

Singly-Linked List



Node contains:

- key
- next pointer

List API

PushFront (Key)

add to front

List API

PushFront (Key)

add to front

Key TopFront ()

return front item

List API

PushFront (Key)	add to front return
Key TopFront ()	front item
PopFront ()	remove front item

List API

PushFront (Key)

add to front

Key TopFront ()

return front item

PopFront ()

remove front item

PushBack (Key)

add to back

also known as Append

List API

PushFront (Key)	add to front
Key TopFront ()	return front item
PopFront ()	remove front item
PushBack (Key)	add to back
Key TopBack ()	return back item

List API

PushFront (Key)

Key TopFront ()

PopFront ()

PushBack (Key)

Key TopBack ()

PopBack ()

add to front

return front item

remove front item

add to back

return back item

remove back item

List API

PushFront (Key)	add to front
Key TopFront ()	return front item
PopFront ()	remove front item
PushBack (Key)	add to back
Key TopBack ()	return back item
PopBack ()	remove back item
Boolean Find (Key)	is key in list?

List API

PushFront (Key)	add to front return
Key TopFront ()	front item
PopFront ()	remove front item
PushBack (Key)	add to back return
Key TopBack ()	back item remove
PopBack ()	back item is key in
Boolean Find (Key)	list?
Erase (Key)	remove key from list

List API

PushFront (Key)

Key TopFront ()

PopFront ()

PushBack (Key)

Key TopBack ()

PopBack ()

Boolean Find (Key)

Erase (Key)

Boolean Empty ()

add to front

return front item

remove front item

add to back

return back item

remove back item

is key in list?

remove key from list

empty list?

List API

<code>PushFront (Key)</code>	add to front
<code>Key TopFront ()</code>	return front item
<code>PopFront ()</code>	remove front item
<code>PushBack (Key)</code>	add to back return
<code>Key TopBack ()</code>	back item remove
<code>PopBack ()</code>	back item is key in
<code>Boolean Find (Key)</code>	list?
<code>Erase (Key)</code>	remove key from list
<code>Boolean Empty ()</code>	empty list?
<code>AddBefore (Node, Key)</code>	adds key before node

List API

<code>PushFront (Key)</code>	add to front
<code>Key TopFront ()</code>	return front item
<code>PopFront ()</code>	remove front item
<code>PushBack (Key)</code>	add to back
<code>Key TopBack ()</code>	return back item
<code>PopBack ()</code>	remove back item
<code>Boolean Find (Key)</code>	is key in list?
<code>Erase (Key)</code>	remove key from list
<code>Boolean Empty ()</code>	empty list?
<code>AddBefore (Node, Key)</code>	adds key before node
<code>AddAfter (Node, Key)</code>	adds key after node

Question

You have an empty list, and then do the following operations:

PushBack(a)

PushFront(b)

PushBack(d)

PushFront(c)

PopBack()

What is the contents of the list now?

c, b, a

c, b, a, d

a, b, c

Question

You have an empty list, and then do the following operations:

PushBack(a)

PushFront(b)

PushBack(d)

PushFront(c)

PopBack()

Here are the list contents after each operation;

PushBack(a) -> a

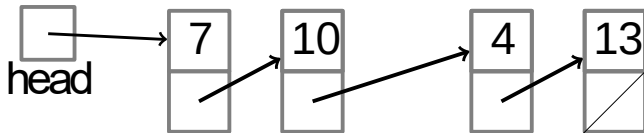
PushFront(b) -> b, a

PushBack(d) -> b, a, d

PushFront(c) -> c, b, a, d

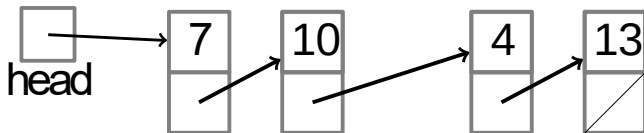
PopBack() -> c, b, a

Times for Some Operations



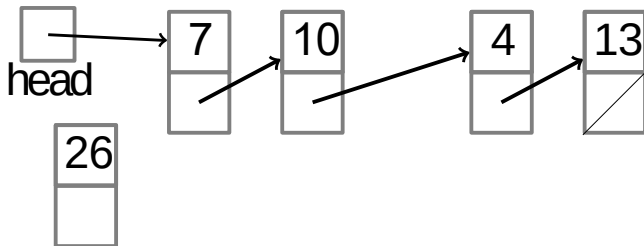
Times for Some Operations

PushFront



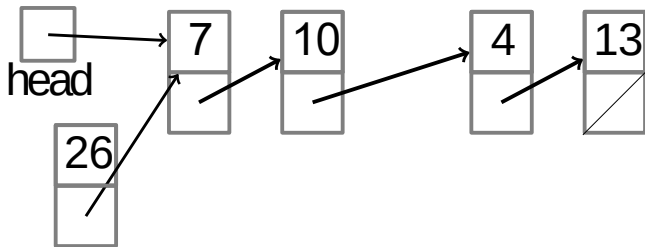
Times for Some Operations

PushFront



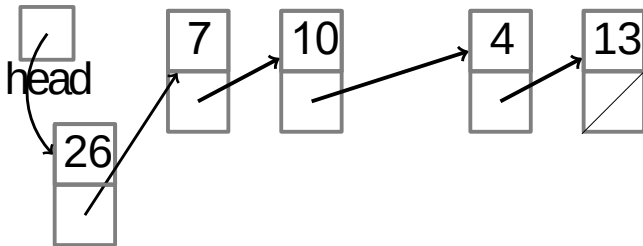
Times for Some Operations

PushFront



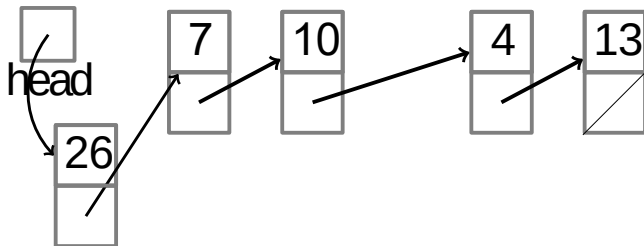
Times for Some Operations

PushFront $O(1)$



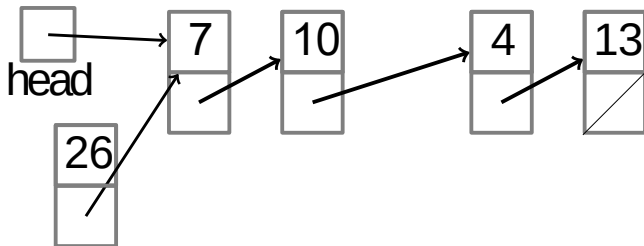
Times for Some Operations

PopFront



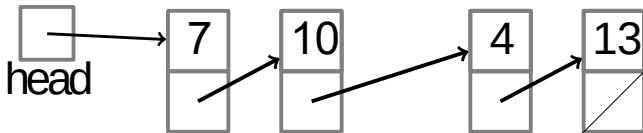
Times for Some Operations

PopFront



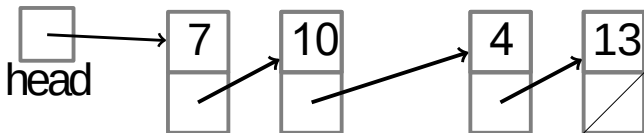
Times for Some Operations

PopFront $O(1)$



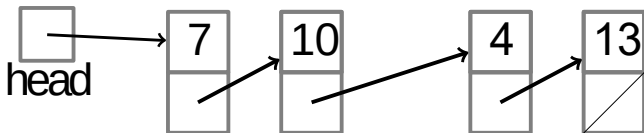
Times for Some Operations

PushBack
(no tail)



Times for Some Operations

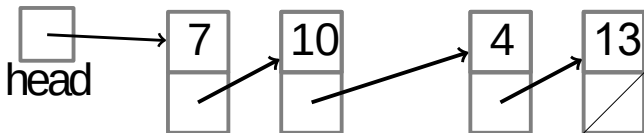
PushBack $O(n)$
(no tail)



Times for Some Operations

PopBack

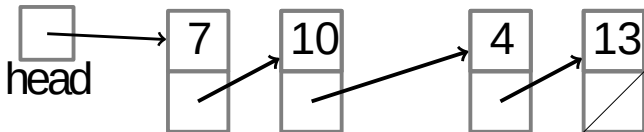
(no tail)



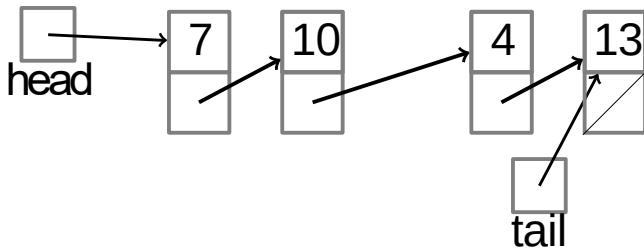
Times for Some Operations

PopBack $O(n)$

(no tail)

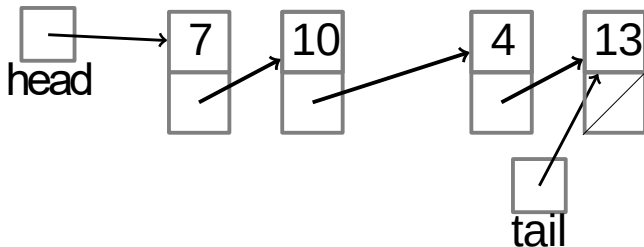


Times for Some Operations



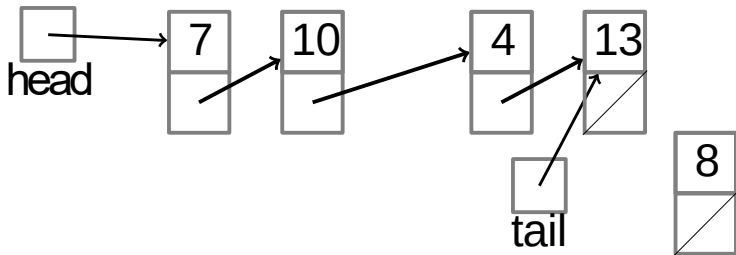
Times for Some Operations

PushBack
(with tail)



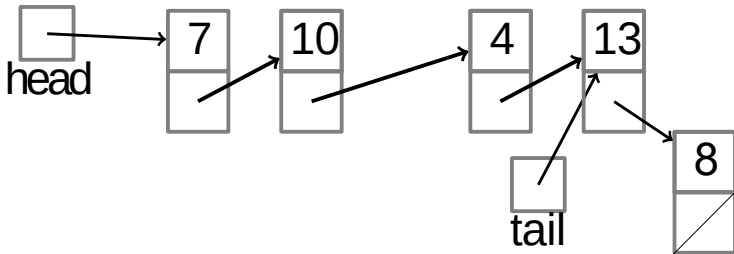
Times for Some Operations

PushBack
(with tail)



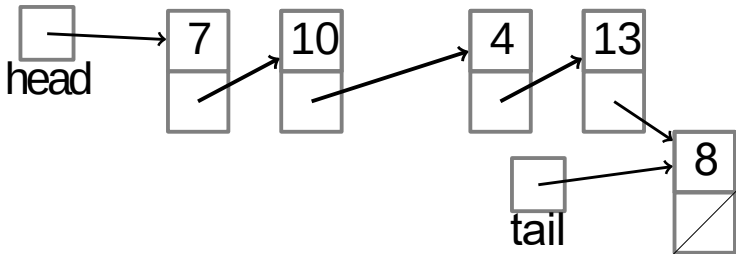
Times for Some Operations

PushBack
(with tail)



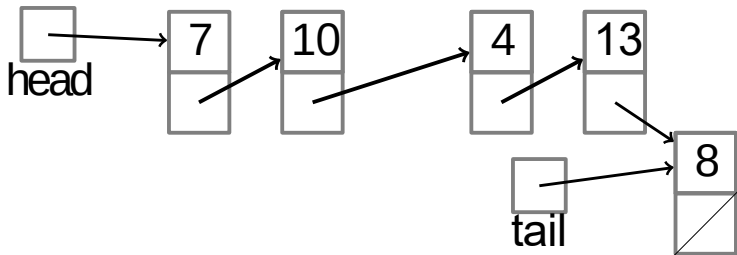
Times for Some Operations

PushBack $O(1)$
(with tail)



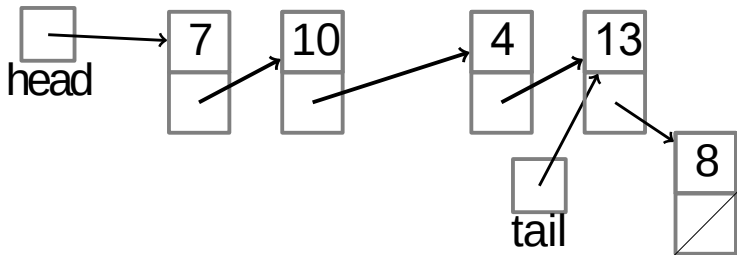
Times for Some Operations

PopBack
(with tail)



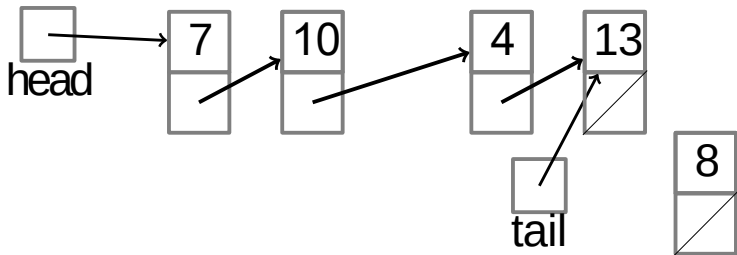
Times for Some Operations

PopBack
(with tail)



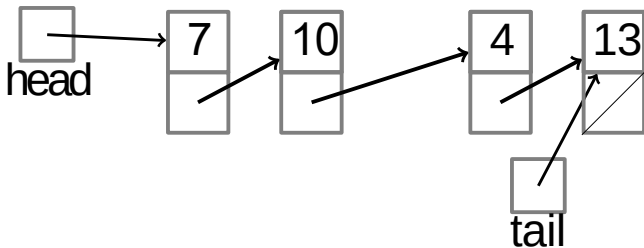
Times for Some Operations

PopBack
(with tail)



Times for Some Operations

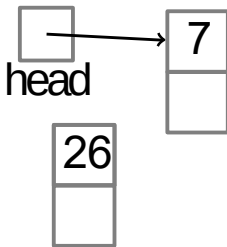
PopBack $O(n)$
(with tail)



Singly-linked List

PushFront(key)

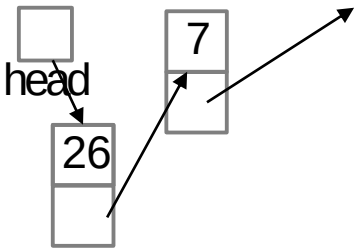
```
node ← new node  
node.key ← key  
node.next ← head  
head ← node  
if tail = nil:  
    tail ← head
```



Singly-linked List

PushFront(key)

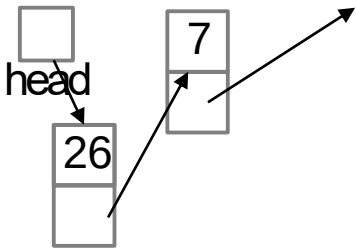
```
node ← new node  
node.key ← key  
node.next ← head  
head ← node  
if tail = nil:  
    tail ← head
```



Singly-linked List

PopFront()

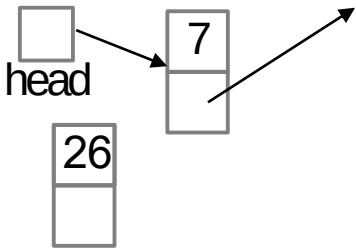
```
if head = nil:  
    ERROR: empty list  
head ← head.next  
if head = nil:  
    tail ← nil
```



Singly-linked List

PopFront()

```
if head = nil:  
    ERROR: empty list  
head ← head.next  
if head = nil:  
    tail ← nil
```



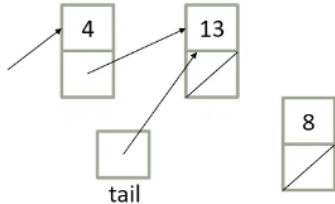
Singly-linked List

PushBack(key)

node $\leftarrow$ new node

node.key $\leftarrow$ *key*

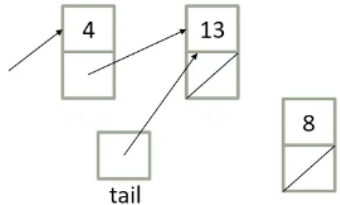
node.next = nil



Singly-linked List

PushBack(key)

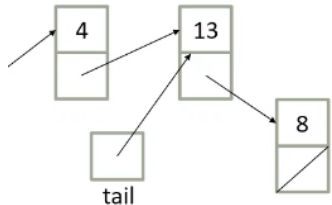
```
node ← new node  
node.key ← key  
node.next = nil  
if tail = nil:  
    head ← tail ← node
```



Singly-linked List

PushBack(key)

```
node ← new node  
node.key ← key  
node.next = nil  
if tail = nil:  
    head ← tail ← node  
else:  
    tail.next ← node  
    tail ← node
```



Singly-linked List

PushBack(*key*)

node $\leftarrow$ new node

node.key $\leftarrow$ *key*

node.next = nil

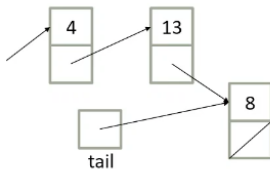
if *tail* = nil:

head $\leftarrow$ *tail* $\leftarrow$ *node*

else:

tail.next $\leftarrow$

node *tail* $\leftarrow$ *node*



Singly-linked List

PopBack()

Singly-linked List

PopBack()

```
if head = nil:  ERROR: empty list
```

Singly-linked List

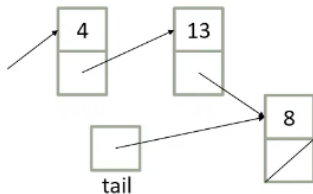
PopBack()

```
if head = nil: ERROR: empty list
if head = tail:
    head  $\leftarrow$  tail  $\leftarrow$  nil
```

Singly-linked List

PopBack()

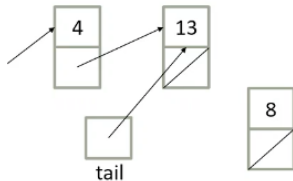
```
if head = nil:    ERROR: empty list
if head = tail :
    head ← tail ← nil
else:
    p ← head
    while p.next.next ≠ nil:
        p ← p.next
```



Singly-linked List

PopBack()

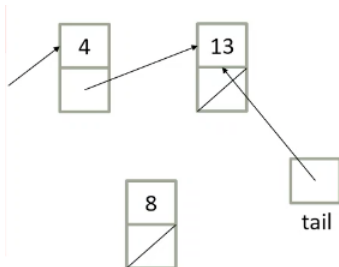
```
if head = nil:    ERROR: empty list
if head = tail :
    head ← tail ← nil
else:
    p ← head
    while p.next.next ≠ nil:
        p ← p.next
    p.next ← nil; tail ← p
```



Singly-linked List

AddAfter(*node*, *key*)

```
node2 ← new node  
node2.key ← key  
node2.next = node.next  
node.next = node2  
if tail = node:  
    tail ← node2
```



Singly-linked List

AddAfter(node, key)

node2 $\leftarrow$ new node

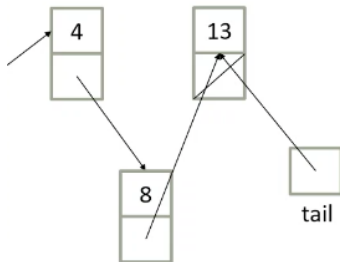
node2.key $\leftarrow$ *key*

node2.next = *node.next*

node.next = *node2*

if *tail* = *node*:

tail $\leftarrow$ *node2*



Singly-Linked List	no tail	with
tail		

PushFront (Key) $O(1)$

Singly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	

Singly-Linked List

no tail

with tail

PushFront (Key)

$O(1)$

TopFront ()

$O(1)$

PopFront ()

$O(1)$

Singly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	
PopFront ()	$O(1)$	
PushBack (Key)	$O(n)$	$O(1)$

Singly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	
PopFront ()	$O(1)$	
PushBack (Key)	$O(n)$	$O(1)$
TopBack ()	$O(n)$	$O(1)$

Singly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	
PopFront ()	$O(1)$	
PushBack (Key)	$O(n)$	$O(1)$
TopBack ()	$O(n)$	$O(1)$
PopBack ()	$O(n)$	

Singly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	
PopFront ()	$O(1)$	
PushBack (Key)	$O(n)$	$O(1)$
TopBack ()	$O(n)$	$O(1)$
PopBack ()	$O(n)$	
Find (Key)	$O(n)$	

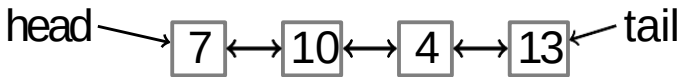
Singly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	
PopFront ()	$O(1)$	
PushBack (Key)	$O(n)$	$O(1)$
TopBack ()	$O(n)$	$O(1)$
PopBack ()	$O(n)$	
Find (Key)	$O(n)$	
Erase (Key)	$O(n)$	

Singly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	
PopFront ()	$O(1)$	
PushBack (Key)	$O(n)$	$O(1)$
TopBack ()	$O(n)$	$O(1)$
PopBack ()	$O(n)$	
Find (Key)	$O(n)$	
Erase (Key)	$O(n)$	
Empty ()	$O(1)$	

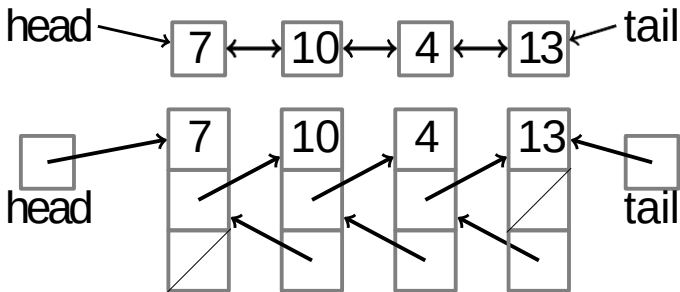
Singly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	
PopFront ()	$O(1)$	
PushBack (Key)	$O(n)$	$O(1)$
TopBack ()	$O(n)$	$O(1)$
PopBack ()	$O(n)$	
Find (Key)	$O(n)$	
Erase (Key)	$O(n)$	
Empty ()	$O(1)$	
AddBefore (Node, Key)	$O(n)$	

Singly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	
PopFront ()	$O(1)$	
PushBack (Key)	$O(n)$	$O(1)$
TopBack ()	$O(n)$	$O(1)$
PopBack ()	$O(n)$	
Find (Key)	$O(n)$	
Erase (Key)	$O(n)$	
Empty ()	$O(1)$	
AddBefore (Node, Key)	$O(n)$	
AddAfter (Node, Key)	$O(1)$	

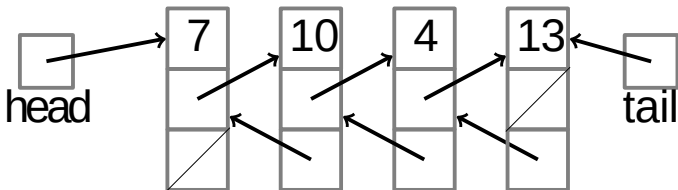
Doubly-Linked List



Doubly-Linked List



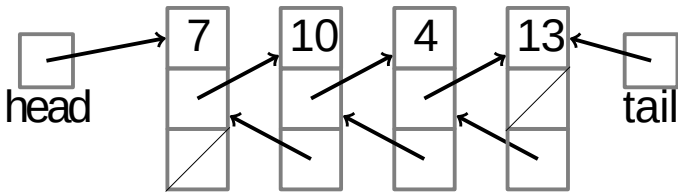
Doubly-Linked List



Node contains:

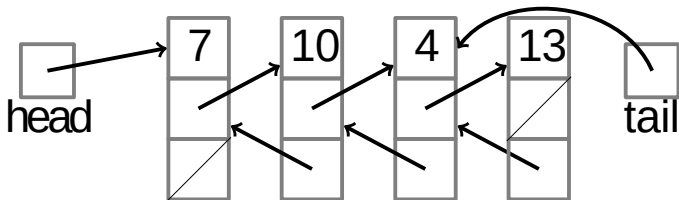
- key
- next pointer
- prev pointer

Doubly-Linked List



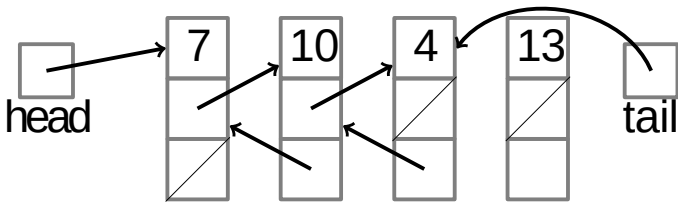
PopBack

Doubly-Linked List



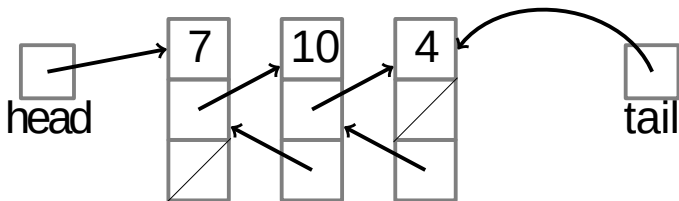
PopBack

Doubly-Linked List



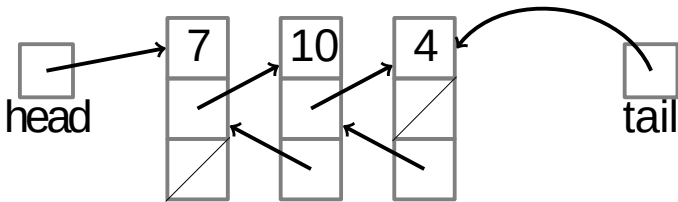
PopBack

Doubly-Linked List



PopBack

Doubly-Linked List



PopBack $O(1)$

Doubly-linked List

PushBack(*key*)

node $\leftarrow$ new node

node.key $\leftarrow$ *key*; *node.next* = nil

Doubly-linked List

PushBack(*key*)

node $\leftarrow$ new node

node.key $\leftarrow$ *key*; *node.next* = nil

if *tail* = nil:

head $\leftarrow$ *tail* $\leftarrow$ *node*

node.prev $\leftarrow$ nil

Doubly-linked List

PushBack(key)

node $\leftarrow$ new node

node.key $\leftarrow$ *key*; *node.next* = nil

if *tail* = nil:

head $\leftarrow$ *tail* $\leftarrow$ *node*

node.prev $\leftarrow$ nil

else:

tail.next $\leftarrow$ *node*

node.prev $\leftarrow$ *tail*

tail $\leftarrow$ *node*

Doubly-linked List

PopBack()

Doubly-linked List

PopBack()

```
if head = nil:  ERROR: empty  
                list
```

Doubly-linked List

PopBack()

```
if head = nil: ERROR: empty list  
if head = tail :  
    head  $\leftarrow$  tail  $\leftarrow$  nil
```

Doubly-linked List

PopBack()

```
if head = nil: ERROR: empty list if  
head = tail :  
    head  $\leftarrow$  tail  $\leftarrow$  nil  
else:  
    tail  $\leftarrow$  tail.prev  
    tail.next  $\leftarrow$  nil
```

Doubly-linked List

AddAfter(node, key)

```
node2 ← new node node2.key ← key  
node2.next ← node.next node2.prev ←  
node node.next ← node2  
if node2.next ≠ nil:  
    node2.next.prev ← node2  
if tail = node: tail ← node2
```

Doubly-linked List

AddBefore(node, key)

node2 ← new node *node2.key*

← *key node2.next* ← *node*

node2.prev ← *node.prev*

node.prev ← *node2*

if *node2.prev* ≠ nil:

node2.prev.next ← *node2*

if *head* = *node*: *head*

 ← *node2*

Singly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	
PopFront ()	$O(1)$	
PushBack (Key)	$O(n)$	$O(1)$
TopBack ()	$O(n)$	$O(1)$
PopBack ()	$O(n)$	
Find (Key)	$O(n)$	
Erase (Key)	$O(n)$	
Empty ()	$O(1)$	
AddBefore(Node, Key)	$O(n)$	
AddAfter (Node, Key)	$O(1)$	

Doubly-Linked List	no tail	with tail
PushFront (Key)	$O(1)$	
TopFront ()	$O(1)$	
PopFront ()	$O(1)$	
PushBack (Key)	$O(n)$	$O(1)$
TopBack ()	$O(n)$	$O(1)$
PopBack ()	$O(n)$ $O(1)$	
Find (Key)	$O(n)$	
Erase (Key)	$O(n)$	
Empty ()	$O(1)$	
AddBefore(Node, Key)	$O(n)$ $O(1)$	
AddAfter (Node, Key)	$O(1)$	

Summary

- Constant time to insert at or remove from the front.

Summary

- Constant time to insert at or remove from the front.
- With tail and doubly-linked, constant time to insert at or remove from the back.

Summary

- Constant time to insert at or remove from the front.
- With tail and doubly-linked, constant time to insert at or remove from the back.
- $O(n)$ time to find arbitrary element.

Summary

- Constant time to insert at or remove from the front.
- With tail and doubly-linked, constant time to insert at or remove from the back.
- $O(n)$ time to find arbitrary
- element. List elements need not be contiguous.

Summary

- Constant time to insert at or remove from the front.
- With tail and doubly-linked, constant time to insert at or remove from the back.
- $O(n)$ time to find arbitrary element.
- List elements need not be contiguous.
- With doubly-linked list, constant time to insert between nodes or remove a node.